

# File Handling in Python

Track: Data Science | Level 1 - Day 17 | Tools: Python 3, Jupyter Notebook

## Level 1 - Day 17: File Handling in Python

Track: Data Science

Task: Task 17 - File I/O and Structured Data Handling

**Objective:** Master Python file I/O: file modes ('r', 'w', 'a'), context managers ('with'), reading/writing CSV files, parsing JSON data, and exception-safe dataset storage.

### 1. Quick Reference & Core Concepts

| Mode / Context     | Syntax               | Data Science Application                                    |
|--------------------|----------------------|-------------------------------------------------------------|
| Read Mode ('r')    | open(filename, 'r')  | Ingesting raw text, TSV, or script files                    |
| Write Mode ('w')   | open(filename, 'w')  | Creating fresh output datasets or overwriting logs          |
| Append Mode ('a')  | open(filename, 'a')  | Streaming live telemetry, training epoch checkpoints        |
| Context Manager    | with open(...) as f: | Guarantees file closure and buffer flushing                 |
| JSON / CSV Modules | import json, csv     | Structured persistence of training configs and tabular data |

### 2. Practical Code Examples

In [code]

```
# Example 1: Safe File Writing and Reading using Context Managers
sample_filename = "training_logs.txt"
with open(sample_filename, "w", encoding="utf-8") as f:
    f.write("Epoch 1 - Loss: 0.4521 - Accuracy: 82.5%")
    f.write("Epoch 2 - Loss: 0.3105 - Accuracy: 89.2%")

with open(sample_filename, "r", encoding="utf-8") as f:
    log_content = f.read()

print(f"Ex 1 - File Read Output: {log_content.strip()}")
```

In [code]

```
# Example 2: Reading Line-by-Line with strip()
with open(sample_filename, "r", encoding="utf-8") as f:
    for line_idx, line in enumerate(f, start=1):
        print(f"Ex 2 - Line {line_idx}: {line.strip()}")
```

In [code]

```
# Example 3: Writing and Reading JSON Configurations
import json
model_meta = {"model_name": "GradientBoosting", "n_estimators": 100, "best_score": 0.942}
json_file = "model_metadata.json"

with open(json_file, "w", encoding="utf-8") as f:
    json.dump(model_meta, f, indent=2)

with open(json_file, "r", encoding="utf-8") as f:
    loaded_meta = json.load(f)

print(f"Ex 3 - Loaded JSON Config: {loaded_meta}")
```

### 3. Applied Data Science Pipeline

#### Applied Data Pipeline: End-to-End CSV Export & Summary Aggregator

In [code]

```
import csv

data_records = [
    ["ID", "Name", "Department", "Salary"],
    ["101", "Aman", "Analytics", 85000],
    ["102", "Pooja", "Engineering", 92000],
    ["103", "Rohan", "Analytics", 78000]
]

csv_file = "employee_summary.csv"
with open(csv_file, "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerows(data_records)

# Read and compute analytics average salary
total_sal = 0
count = 0
with open(csv_file, "r", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for row in reader:
        total_sal += float(row["Salary"])
        count += 1

print(f"Processed {count} employees from {csv_file}. Average Salary: ${total_sal / count:,.2f}")
```